TRƯỜNG ĐẠI HỌC .....
KHOA .....

BÁO CÁO NGHIÊN CỨU

KHẢO SÁT ẢNH HƯỞNG CỦA LƯỢNG TỬ HÓA SAU HUẤN LUYỆN INT8 ĐẾN ĐỘ CHÍNH XÁC VÀ DUNG LƯỢNG MÔ HÌNH NHẬN DIỆN HOẠT ĐỘNG TỪ CẢM BIẾN UCI-HAR

Mô hình ResNetv1-6 một chiều: luồng thuật toán, trích xuất tham số và cài đặt suy luận bằng ngôn ngữ C

Nhóm thực hiện: Nhóm 3 sinh viên

Sinh viên 1: ..... MSSV: .....

Sinh viên 2: ..... MSSV: .....

Sinh viên 3: ..... MSSV: .....

Giảng viên hướng dẫn: .....

Mã nguồn và kết quả: `https://github.com/Royce-23/quantization`

Tháng 10 năm 2026

Tóm tắt

Báo cáo khảo sát ảnh hưởng của lượng tử hóa sau huấn luyện (PTQ) INT8 đến độ chính xác và dung lượng của mô hình nhận diện hoạt động người từ cảm biến quán tính trên bộ dữ liệu UCI-HAR. Nhóm huấn luyện baseline FP32 theo kiến trúc ResNetv1-6 một chiều (10.726 tham số), đạt accuracy 92,60% và macro-F1 0,9271 trên tập test chính thức; sau đó lượng tử hóa full-integer INT8 bằng TensorFlow Lite với ba kích thước calibration (50, 200, 500 cửa sổ) và ba seed, chọn cấu hình bằng validation trước khi mở test. Trên test, INT8 ($n = 500$) đạt 92,1841 $\pm$ 0,2722%, thấp hơn FP32 0,4185 điểm phần trăm, trong khi file mô hình giảm 51,47% (51.728 $\rightarrow$ 25.104 byte, 2,06 lần). Theo góp ý của giảng viên hướng dẫn, báo cáo trình bày luồng và thuật toán của từng khối bằng công thức, trích xuất toàn bộ tham số (kèm bảng thống kê và giá trị), và cài đặt suy luận bằng C99 thuần, khớp với Keras trên 12 cửa sổ đối chiếu (sai số logits cỡ 10^{-6}, argmax 12/12). Phạm vi chỉ gồm thí nghiệm trên máy tính; báo cáo không đo RAM, Flash, độ trễ hay năng lượng trên vi điều khiển.

Mục lục

1 Giới thiệu **3**

1.1 Bối cảnh và động cơ 3
1.2 Mục tiêu 3
1.3 Phạm vi và giới hạn 3
1.4 Tổng quan quy trình và bố cục báo cáo 3

2 Cơ sở lý thuyết **4**

2.1 Nhận diện hoạt động từ cảm biến và các phép toán của mạng 4
2.2 Kết nối residual 5
2.3 Lượng tử hóa affine và PTQ INT8 5
2.4 Chỉ số đánh giá 6

3 Dữ liệu và tiền xử lý **6**

3.1 Bộ dữ liệu UCI-HAR 6
3.2 Chia tập 6
3.3 Chuẩn hóa Z-score 6

4 Mô hình ResNetv1-6 một chiều: luồng và thuật toán **7**

4.1 Tổng quan 7
4.2 Luồng xử lý có nhánh 7
4.3 Thuật toán từng khối 7
4.4 Số tham số và số phép tính 9

5 Phương pháp thực nghiệm **10**

5.1 Huấn luyện baseline FP32 . . . 10
5.2 PTQ full-integer INT8 . . . 10
5.3 Quy trình chọn cấu hình và đánh giá . . . 10
5.4 Trích xuất tham số và sinh mã C . . . 11
5.5 Cài đặt suy luận bằng C99 . . . 11

6 Kết quả và thảo luận **12**
6.1 Baseline FP32 . . . 12
6.2 Ảnh hưởng của kích thước calibration (validation) . . . 12
6.3 So sánh FP32 và INT8 trên test . . . 13
6.4 Dung lượng mô hình . . . 13
6.5 Kết quả trích xuất tham số . . . 14
6.6 Kết quả suy luận bằng C . . . 15
6.7 Thảo luận . . . 16

7 Kết luận và hướng phát triển **16**

A Kernel đầy đủ của lớp chiếu nhánh B **18**

B Tái lập kết quả **18**

1 Giới thiệu

1.1 Bối cảnh và động cơ

Nhận diện hoạt động người (Human Activity Recognition, HAR) từ cảm biến gia tốc và con quay hồi chuyển là bài toán nền tảng của thiết bị đeo và theo dõi sức khỏe. Để chạy trực tiếp trên vi điều khiển, mô hình phải nhỏ và dùng ít bộ nhớ. Lượng tử hóa đưa trọng số và giá trị kích hoạt từ số thực 32 bit (FP32) xuống số nguyên 8 bit (INT8), về nguyên tắc giảm kích thước khoảng bốn lần, đổi lại sai số làm tròn có thể làm giảm độ chính xác. Bài báo nền của đề tài [1] khảo sát lượng tử hóa và triển khai mạng nơ-ron trên vi điều khiển, trong đó ResNetv1-6 cho UCI-HAR là một trường hợp cụ thể (Hình 4 và mục 6.1.1 của bài báo).

1.2 Mục tiêu

1. Xây dựng và huấn luyện baseline FP32 ResNetv1-6 một chiều trên UCI-HAR, bám theo cấu hình của bài báo nền.
2. Lượng tử hóa sau huấn luyện full-integer INT8 bằng TensorFlow Lite, khảo sát ảnh hưởng của số cửa sổ calibration (50, 200, 500) với ba seed.
3. So sánh FP32 và INT8 theo accuracy, macro-F1, ma trận nhầm lẫn và dung lượng file mô hình.
4. Theo góp ý của giảng viên hướng dẫn: (i) mô tả *luồng và thuật toán* của mô hình ở mức từng khối, từng nhánh, bằng công thức; (ii) *trích xuất tham số* đã huấn luyện và trình bày kết quả; (iii) *viết mã C* thực thi thuật toán suy luận và kiểm chứng với mô hình gốc.

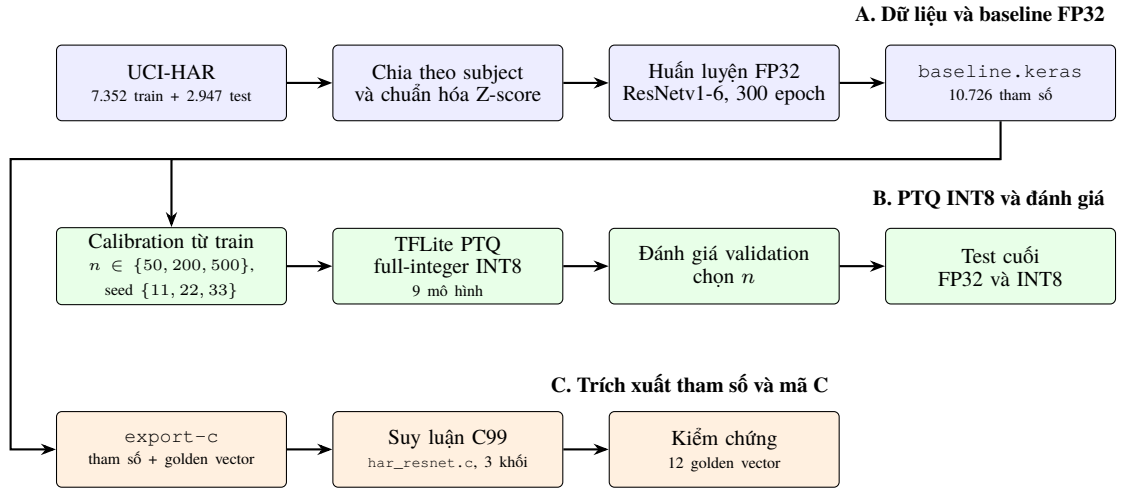
1.3 Phạm vi và giới hạn

Đề tài là nghiên cứu thực nghiệm trên máy tính, một bộ dữ liệu, một kiến trúc và một kỹ thuật PTQ. Chỉ số chính là accuracy, macro-F1 và dung lượng file; ma trận nhầm lẫn hỗ trợ phân tích. Báo cáo *không* đo RAM, Flash, độ trễ hoặc năng lượng trên vi điều khiển, và *không* suy các đại lượng đó từ phép đo trên laptop. Mã C hiện cài đặt suy luận FP32; phần INT8 chỉ được thực nghiệm bằng interpreter của TensorFlow Lite trên Python.

Về tên gọi: mạng trong báo cáo là **ResNetv1-6** theo ký hiệu của bài báo nền; "6" ứng với sáu lớp có trọng số (năm lớp Conv1D và một lớp Dense). Mạng này không có BatchNormalization ở bất kỳ lớp nào.

1.4 Tổng quan quy trình và bố cục báo cáo

Hình 1 tóm tắt ba nhóm công việc: (A) dữ liệu và baseline FP32; (B) PTQ INT8 và đánh giá; (C) trích xuất tham số và suy luận bằng C. Mục 2 trình bày cơ sở lý thuyết và công thức; Mục 3 mô tả dữ liệu; Mục 4 mô tả luồng và thuật toán của mô hình; Mục 5 trình bày phương pháp thực nghiệm; Mục 6 trình bày kết quả, gồm kết quả trích xuất tham số; Mục 7 kết luận.



Hình 1. Ba nhóm công việc của đề tài. Cả PTQ INT8 và việc xuất tham số cho mã C đều xuất phát từ cùng một checkpoint `baseline.keras`.

2 Cơ sở lý thuyết

2.1 Nhận diện hoạt động từ cảm biến và các phép toán của mạng

Đầu vào của mô hình là một cửa sổ tín hiệu $x \in \mathbb{R}^{L \times C}$ (L mốc thời gian, C kênh). Các phép toán dưới đây đủ để mô tả toàn bộ mạng; ký hiệu dùng xuyên suốt báo cáo và khớp từng dòng với mã Python tham chiếu và mã C.

Conv1D, padding *same*, stride 1, kernel lẻ K, có bias. Với kernel $W \in \mathbb{R}^{K \times C_{in} \times C_{out}}$ và bias $b \in \mathbb{R}^{C_{out}}$:

$$y_{t,c_o} = b_{c_o} + \sum_{k=0}^{K-1} \sum_{c_i=0}^{C_{in}-1} W_{k,c_i,c_o}\, \tilde{x}_{t+k-p,\,c_i}, \qquad p = \lfloor K/2 \rfloor, \tag{1}$$

trong đó $\tilde{x}$ là tín hiệu được đệm không:

$$\tilde{x}_{s,c} = \begin{cases} x_{s,c}, & 0 \le s < L, \\ 0, & \text{ngược lại.} \end{cases} \tag{2}$$

Độ dài giữ nguyên ($L_{out} = L$). Với $K = 1$ thì $p = 0$, không đệm gì, trùng với padding *valid*; vì vậy lớp chiếu $K = 1$ dùng chung công thức (1).

ReLU, MaxPool và GlobalMaxPool.

$$\mathrm{ReLU}(x)_{t,c} = \max(0, x_{t,c}), \tag{3}$$

$$\mathrm{MaxPool}_2(x)_{t,c} = \max\big(x_{2t,c},\, x_{2t+1,c}\big), \quad t = 0, \ldots, \lfloor L/2 \rfloor - 1, \tag{4}$$

$$g_c = \max_{0 \le t < L} x_{t,c} \quad \text{(GlobalMaxPool)}. \tag{5}$$

Dense và dự đoán. Với vector $g \in \mathbb{R}^{D_{in}}$, lớp Dense không activation cho *logits* thô, và nhãn

dự đoán là chỉ số lớn nhất:

$$z_o = b_o + \sum_{i=0}^{D_{in}-1} g_i\, W_{i,o}, \qquad \hat{y} = \arg\max_o \; z_o. \tag{6}$$

Softmax $p_o = e^{z_o - m} / \sum_j e^{z_j - m}$ với $m = \max_j z_j$ không nằm trong mô hình vì hàm mất mát huấn luyện dùng `from_logits=True`; do arg max z = arg max p nên suy luận chỉ cần arg max.

2.2 Kết nối residual

Một khối residual [3] cộng đầu ra của nhánh chính $F(x)$ với nhánh tắt (shortcut) $S(x)$ rồi qua ReLU:

$$y = \mathrm{ReLU}\big(F(x) + S(x)\big). \tag{7}$$

Phép cộng đòi hỏi $F(x)$ và $S(x)$ cùng shape. Khi nhánh chính đổi số kênh hoặc độ dài thời gian, nhánh tắt phải là *projection shortcut* (Conv1D $K = 1$ rồi pooling cùng bước); khi shape không đổi, nhánh tắt có thể là *identity* $S(x) = x$, không tốn tham số. Đây chính là lý do Khối 1 và Khối 2 của mạng có hai kiểu nhánh khác nhau (Mục 4).

2.3 Lượng tử hóa affine và PTQ INT8

Lượng tử hóa affine ánh xạ số thực r sang số nguyên q bằng $scale\ S > 0$ và $zero\text{-}point\ Z$ [4, 5]:

$$q = \mathrm{clip}\Big(\mathrm{round}\big(r/S + Z\big),\, -128,\, 127\Big), \qquad \hat{r} = S\,(q - Z). \tag{8}$$

Với phép tích chập INT8 theo dạng tổng quát của TFLite (trọng số lượng tử hóa theo từng kênh ra, $Z_w = 0$), tích lũy dùng số nguyên 32 bit rồi *requantize*:

$$\mathrm{acc}_{t,c_o} = \sum_{k,c_i} \big(q^x_{t+k-p,c_i} - Z_x\big)\, q^w_{k,c_i,c_o} + q^b_{c_o}, \qquad q^y_{t,c_o} = \mathrm{clip}\Big(Z_y + \mathrm{round}\big(M_{c_o}\,\mathrm{acc}_{t,c_o}\big), -128, 127\Big), \tag{9}$$

với $M_{c_o} = S_x S_{w,c_o} / S_y$, được hiện thực bằng một cặp (multiplier số nguyên, shift) thay cho số thực. Công thức (9) chỉ để tham khảo; mã C trong repository hiện chưa cài INT8.

PTQ so với QAT. PTQ lượng tử hóa mô hình đã huấn luyện, chỉ cần một tập nhỏ dữ liệu *calibration* để ước lượng miền giá trị của activation (từ đó suy ra S, Z), không huấn luyện lại. QAT mô phỏng lượng tử hóa trong lúc huấn luyện và thường giữ độ chính xác tốt hơn. Bài báo nền dùng luồng MicroAI/QAT; đề tài này chủ động khảo sát *TFLite PTQ full-integer INT8*, nên kết quả là thí nghiệm dựa trên bài báo, không tuyên bố tái hiện nguyên bản.

Tập	Số subject	Số cửa sổ
Train (huấn luyện)	17	5.867
Validation	4	1.485
Test chính thức	9	2.947
Tổng	30	10.299

Bảng 1. Chia dữ liệu theo subject (nguồn: `data/processed/split.json`).

2.4 Chỉ số đánh giá

Với ma trận nhầm lẫn M (M_{ij}: số mẫu nhãn thật i được dự đoán j), Acc $= \sum_i M_{ii} / \sum_{ij} M_{ij}$, $P_c = M_{cc} / \sum_i M_{ic}$, $R_c = M_{cc} / \sum_j M_{cj}$, $F1_c = 2P_cR_c/(P_c + R_c)$ và

$$\text{macro-F1} = \frac{1}{6} \sum_{c=1}^{6} F1_c. \tag{10}$$

Độ lệch chuẩn qua các seed là độ lệch chuẩn mẫu (chia $n - 1$). Hệ số nén dung lượng là dung lượng FP32/dung lượng INT8.

3 Dữ liệu và tiền xử lý

3.1 Bộ dữ liệu UCI-HAR

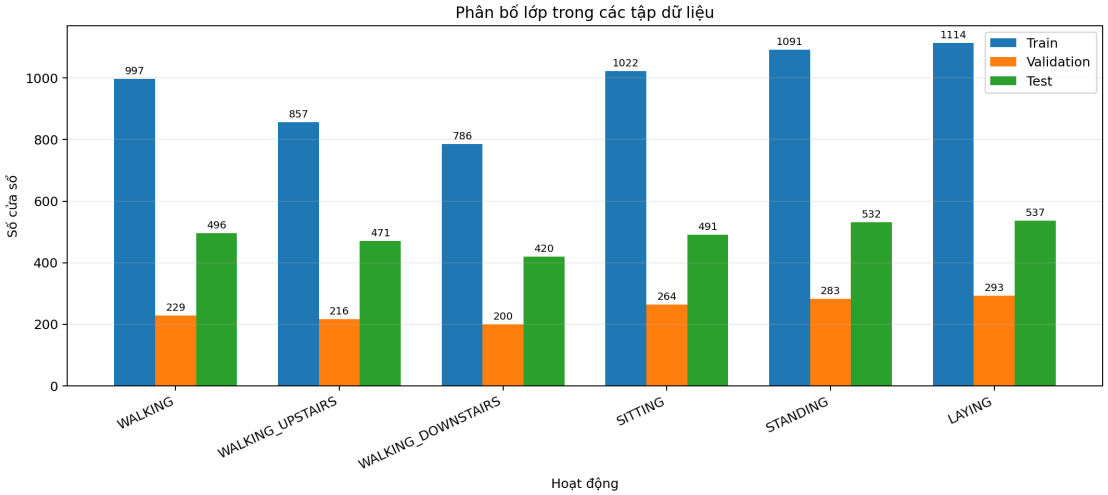
UCI-HAR [2] gồm tín hiệu quán tính của 30 tình nguyện viên thực hiện sáu hoạt động: `WALKING`, `WALKING_UPSTAIRS`, `WALKING_DOWNSTAIRS`, `SITTING`, `STANDING`, `LAYING`. Tín hiệu được lấy mẫu ở 50 Hz và cắt thành cửa sổ dài 2,56 giây, tức 128 mốc thời gian. Đề tài dùng *tín hiệu thô theo cửa sổ* thay cho 561 đặc trưng tính sẵn, đúng như mục 6.1.1 của bài báo nền. Mỗi mẫu là một tensor $(128, 9)$ theo thứ tự `[time, channel]`; chín kênh được cố định là: `total_acc_x/y/z`, `body_gyro_x/y/z`, `body_acc_x/y/z`.

3.2 Chia tập

Tập test là tập test chính thức của UCI-HAR (9 subject). Từ tập train chính thức (21 subject), nhóm tách các subject 27–30 làm validation để chọn checkpoint và cấu hình calibration mà không để cùng một người xuất hiện ở cả train và validation (Bảng 1).

3.3 Chuẩn hóa Z-score

Mỗi kênh được chuẩn hóa $x^n_{t,c} = (x^{raw}_{t,c} - \mu_c)/\sigma_c$, với μ_c, σ_c *chỉ* tính trên 5.867 cửa sổ train mô hình, không dùng validation hay test. Các giá trị μ_c, σ_c được lưu và dùng lại trong mã C (Bảng 12). Hình 2 cho thấy phân bố lớp của các tập.



Hình 2. Phân bố nhãn theo tập (đầu ra của `python main.py visualize-tv1`).

Nhánh	Xuất phát từ	Phép toán (theo thứ tự)	Hợp lại tại	Shape khi hợp
A_1 (chính, Khối 1)	x^n $(128, 9)$	Conv K=3 $\to$ ReLU $\to$ Conv K=3 $\to$ MaxPool/2	$\oplus$ Khối 1	$(64, 32)$
B_1 (projection)	x^n (cùng nguồn A_1)	Conv K=1 (9$\to$32 kênh) $\to$ MaxPool/2	$\oplus$ Khối 1	$(64, 32)$
A_2 (chính, Khối 2)	$x^{(1)}$ $(64, 32)$	Conv K=3 $\to$ ReLU $\to$ Conv K=3	$\oplus$ Khối 2	$(64, 32)$
B_2 (identity)	$x^{(1)}$ (cùng nguồn A_2)	không có (truyền thẳng)	$\oplus$ Khối 2	$(64, 32)$

Bảng 2. Các nhánh của mô hình. Hai nhánh của mỗi khối cùng xuất phát từ *cùng một tensor*, không phải nhánh này nối tiếp nhánh kia.

4 Mô hình ResNetv1-6 một chiều: luồng và thuật toán

4.1 Tổng quan

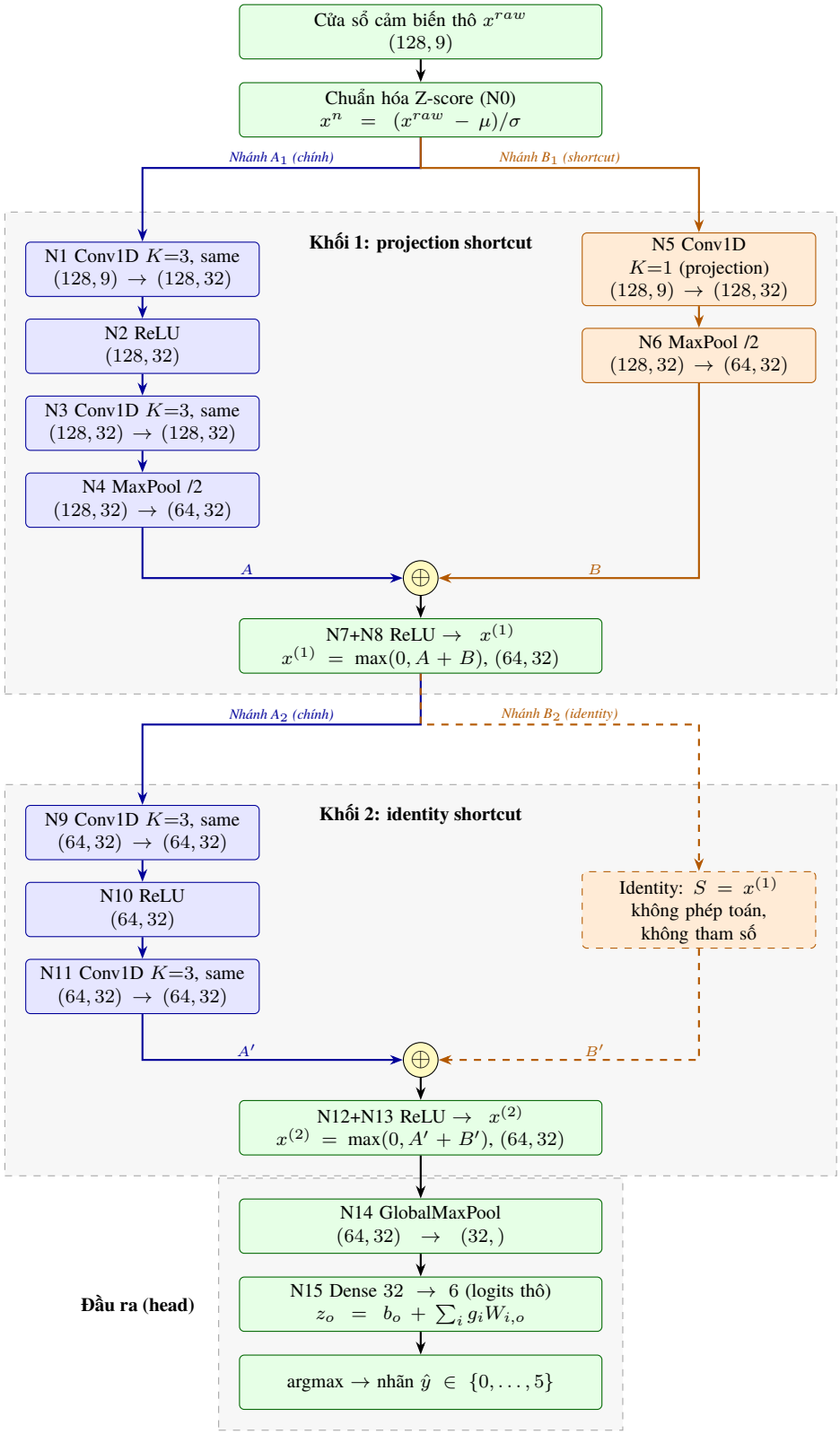
Mô hình gồm hai khối residual với 32 filter, tiếp theo là GlobalMaxPooling và lớp Dense 6 đầu ra. Tổng cộng 10.726 tham số, 6 lớp có trọng số. Khối 1 nhận thẳng cửa sổ đã chuẩn hóa (không có lớp stem riêng), đổi $(128, 9) \to (64, 32)$ và dùng *projection shortcut*; Khối 2 giữ $(64, 32) \to (64, 32)$ và dùng *identity shortcut*.

4.2 Luồng xử lý có nhánh

Hình 3 thể hiện luồng suy luận. Có hai *điểm rẽ nhánh*: sau bước chuẩn hóa (vào Khối 1) và sau $x^{(1)}$ (vào Khối 2); mỗi điểm rẽ nhánh cho hai nhánh cùng nguồn, hợp lại tại một node cộng $\oplus$ rồi qua ReLU. Bảng 2 liệt kê từng nhánh: bắt đầu từ đâu, đi qua phép toán nào, hợp lại ở đâu.

4.3 Thuật toán từng khối

Dùng các phép toán (1)–(7), ký hiệu $\text{Conv}_K(\cdot; W, b)$ là công thức (1) với kernel K.



Hình 3. Luồng suy luận của ResNetv1-6 một chiều. Màu xanh: nhánh chính A (có trọng số); màu cam: nhánh tắt B (projection ở Khối 1, identity nét đứt ở Khối 2); $\oplus$: điểm hợp nhánh bằng phép cộng phần tử. Nhãn N0–N15 là chỉ số node dùng xuyên suốt tài liệu và mã.

Node	Phép toán	Công thức	Hàm C (`firmware/src/har_resnet.c`)	Tham số
N0	Z-score	Mục 3.3	`har_normalize`	18 hằng số
N1	Conv1D K=3, 9→32	(1), p=1	`har_conv1d_same` (trong `har_block1_branch_main`)	896
N2	ReLU	(3)	`har_relu`	0
N3	Conv1D K=3, 32→32	(1), p=1	`har_conv1d_same` (`har_block1_branch_main`)	3.104
N4	MaxPool/2	(4)	`har_maxpool1d_2`	0
N5	Conv1D K=1, 9→32	(1), p=0	`har_conv1d_same` (`har_block1_branch_shortcut`)	320
N6	MaxPool/2	(4)	`har_maxpool1d_2`	0
N7	Cộng $A + B$	(7)	`har_add` (trong `har_residual_merge_relu`)	0
N8	ReLU sau cộng	(3), (7)	`har_relu` (`har_residual_merge_relu`)	0
N9	Conv1D K=3, 32→32	(1), p=1	`har_conv1d_same` (`har_block2_branch_main`)	3.104
N10	ReLU	(3)	`har_relu`	0
N11	Conv1D K=3, 32→32	(1), p=1	`har_conv1d_same` (`har_block2_branch_main`)	3.104
N12	Cộng $A' + x^{(1)}$ (identity)	(7)	`har_add` (`har_residual_merge_relu`)	0
N13	ReLU sau cộng	(3), (7)	`har_relu` (`har_residual_merge_relu`)	0
N14	GlobalMaxPool	(5)	`har_global_max_pool` (`har_head_forward`)	0
N15	Dense $32 \to 6$	(6)	`har_dense` (`har_head_forward`)	198
–	argmax	(6)	`har_argmax`	0

Bảng 3. Ánh xạ từng node của Hình 3 sang công thức, hàm C và số tham số. Tổng tham số có trọng số: $896 + 3.104 + 320 + 3.104 + 3.104 + 198 = 10.726$.

Khối 1 (nhận $x^n \in \mathbb{R}^{128\times 9}$, cho $x^{(1)} \in \mathbb{R}^{64\times 32}$):

$$A = \text{MaxPool}_2\Big(\text{Conv}_3\big(\text{ReLU}(\text{Conv}_3(x^n; W_1, b_1)); W_2, b_2\big)\Big), \tag{11}$$

$$B = \text{MaxPool}_2\big(\text{Conv}_1(x^n; W_p, b_p)\big), \tag{12}$$

$$x^{(1)} = \text{ReLU}(A + B). \tag{13}$$

Khối 2 (nhận $x^{(1)}$, cho $x^{(2)} \in \mathbb{R}^{64\times 32}$):

$$A' = \text{Conv}_3\big(\text{ReLU}(\text{Conv}_3(x^{(1)}; W_3, b_3)); W_4, b_4\big), \qquad B' = x^{(1)}, \tag{14}$$

$$x^{(2)} = \text{ReLU}(A' + B'). \tag{15}$$

Đầu ra: $g = \text{GlobalMaxPool}(x^{(2)}) \in \mathbb{R}^{32}$, $z = g\,W_d + b_d \in \mathbb{R}^6$ theo (6), $\hat{y} = \arg\max_o z_o$.

4.4 Số tham số và số phép tính

Số tham số: Conv1D có $KC_{in}C_{out} + C_{out}$, Dense có $D_{in}D_{out} + D_{out}$. Cộng sáu lớp: $896 + 3.104 + 320 + 3.104 + 3.104 + 198 = 10.726$. Số phép nhân–cộng (MAC) của một lớp Conv1D, có trừ các vị trí kernel rơi ra ngoài biên do zero-padding, là

$$\text{MAC} = \big(KL - p(p+1)\big)\, C_{in}\, C_{out}, \tag{16}$$

cho tổng 927.360 MAC mỗi cửa sổ (chặn trên không trừ biên: 934.080).

Siêu tham số	Giá trị
Bộ tối ưu	SGD, momentum 0,9
Batch size / số epoch	64 / 300
Learning rate ban đầu	0,05; nhân 0,13 tại epoch 100, 200, 250
Weight decay	5×10^{-4}
Mixup	$\alpha = 0{,}2$ (bài báo không công bố α)
Số filter	32

Bảng 4. Cấu hình huấn luyện baseline (nguồn: `configs/project.json`).

5 Phương pháp thực nghiệm

5.1 Huấn luyện baseline FP32

Cấu hình bám mục 6.1.1 của bài báo nền (Bảng 4). Hàm mất mát là cross-entropy với `from_logits=True`; checkpoint được chọn theo validation accuracy cao nhất. Môi trường: Python 3.10.12, TensorFlow CPU 2.16.1, NumPy 1.26.4, seed 42. Mixup [6] trộn hai mẫu $(x_i, y_i), (x_j, y_j)$ thành $\lambda x_i + (1-\lambda)x_j$ với nhãn tương ứng, $\lambda \sim \text{Beta}(\alpha, \alpha)$.

Các điểm khác bài báo (nêu rõ để không hiểu nhầm là tái hiện nguyên bản): (i) tách subject 27–30 làm validation; (ii) $\alpha = 0{,}2$ do bài báo không công bố; (iii) nhánh INT8 là TFLite PTQ thay vì MicroAI/QAT; (iv) kết quả hiện tại là một lần chạy seed 42, còn bài báo báo trung bình 15 lần chạy.

5.2 PTQ full-integer INT8

Nguồn mô hình duy nhất là checkpoint FP32 (SHA-256 bắt đầu bằng `aa323b3e55d1`; đã kiểm chứng trùng với file thực). Quy trình trong `src/quantize.py`:

1. Nạp `baseline.keras`, kiểm tra SHA-256 với metadata; xuất `baseline_fp32.tflite` để so sánh cùng định dạng file.
2. Với mỗi kích thước calibration $n \in \{50, 200, 500\}$ và seed $\{11, 22, 33\}$, lấy n cửa sổ *không lặp* từ x_{train} (đã Z-score; không chuẩn hóa lại, không dùng validation/test).
3. Representative dataset cho TFLite Converter quan sát miền giá trị của activation, suy ra S và Z theo (8).
4. Ép tập toán tử `TFLITE_BUILTINS_INT8`, đầu vào và đầu ra kiểu `int8`.
5. Mở lại từng mô hình bằng interpreter, kiểm tra mọi tensor; nếu còn tensor float thì dừng thay vì gọi nhầm là full-integer.

Kết quả là một mô hình FP32 TFLite và chín mô hình INT8 (3×3).

5.3 Quy trình chọn cấu hình và đánh giá

FP32 và cả chín mô hình INT8 được chạy trên validation; gom ba seed theo từng n, chọn n có accuracy trung bình cao nhất (hòa thì ưu tiên macro-F1 trung bình rồi n nhỏ hơn). Chỉ *sau khi chốt* n mới mở test và đánh giá FP32 cùng ba seed của n đã chọn. FP32 TFLite và INT8 dùng đúng cùng tensor đã chuẩn hóa, cùng thứ tự nhãn. Cách này tránh dùng test để chọn cấu hình.

5.4 Trích xuất tham số và sinh mã C

Lệnh `python main.py export-c` (`src/export_c.py`) thực hiện:

1. Kiểm tra SHA-256 của `baseline.keras` với `checkpoint_id` trong metadata; lệch thì dừng.
2. Nạp mô hình, lấy trọng số *theo tên layer*, kiểm tra shape từng layer và tổng $= 10.726$ khớp `model.count_params()`.
3. Ghi `models/params_fp32.npz` gồm 14 khóa (mỗi lớp có `_w` và `_b`, cùng `input_mean` và `input_std`) và file `params_manifest.json`.
4. Sinh mã C tĩnh gồm hai cặp tệp `.h` và `.c`: `har_resnet_params` (mảng `const float`) và `har_golden` (12 cửa sổ đối chiếu lấy từ `sanity_samples.npz`, kèm logits của Keras).

Quy ước bố cục: tensor kích hoạt `[time][channel]`; kernel Conv1D `[k][cin][cout]`, đúng thứ tự C-order khi `reshape(-1)` mảng Keras, *không transpose*; Dense `[din][dout]`. Số thực in dạng `{:.9g}f` để chuyển float32 sang văn bản rồi đọc lại không mất bit. Lệnh `python main.py describe-params` sinh `results/parameters_reference.md` mô tả từng tham số (vai trò, nhánh, shape từng chiều, công thức, tên biến C, thống kê).

5.5 Cài đặt suy luận bằng C99

`firmware/src/har_resnet.c` viết tay theo công thức ở Mục 2: sáu hàm phép toán (`har_conv1d_same`, `har_relu`, `har_maxpool1d_2`, `har_add`, `har_global_max_pool`, `har_dense`) và ba hàm theo khối `har_block1_forward`, `har_block2_forward`, `har_head_forward`, ánh xạ 1–1 với Hình 3. Mã không dùng `malloc`, mảng độ dài biến (VLA) hay `double`; biên dịch C99 với `-Wall -Wextra -Wpedantic` không cảnh báo. Toàn bộ buffer trung gian là năm mảng `static float`: hai mảng 128×32 và ba mảng 64×32 (57.344 byte). Do dùng buffer tĩnh, hàm *không an toàn đa luồng*: chỉ một lời gọi suy luận chạy tại một thời điểm.

Mã 1. Phép Conv1D same trong `har_resnet.c` (đã lược bớt chú thích tiếng Việt).

```
void har_conv1d_same(const float *in, int len, int cin,
                     const float *w, const float *b,
                     int ksize, int cout, float *out) {
   int pad = ksize / 2;
   int t;

   for (t = 0; t < len; t++) {
      int co;
      for (co = 0; co < cout; co++) {
         float acc = b[co];
         int ki;
         for (ki = 0; ki < ksize; ki++) {
            int ts = t + ki - pad;
            int ci;
            if (ts < 0 || ts >= len) {
               continue;
            }
            for (ci = 0; ci < cin; ci++) {
               acc += in[ts * cin + ci] * w[(ki * cin + ci) * cout + co];
            }
         }
         out[t * cout + co] = acc;
      }
   }
}
```

Mã 2. Ba hàm khối và hàm suy luận (đã lược bớt chú thích tiếng Việt). Hàm `har_resnet_predict` gọi tuần tự Khối 1, Khối 2, Head trên cùng năm buffer tĩnh.

```
void har_block1_forward(const float *x_norm, float *y64,
                  float *s128_0, float *s128_1, float *s64_0) {
   har_block1_branch_main(x_norm, s128_0, s128_1, y64); /* A -> y64 */
   har_block1_branch_shortcut(x_norm, s128_0, s64_0); /* B -> s64_0 */
   har_residual_merge_relu(y64, s64_0, 64 * HAR_FILTERS, y64); /* add(A,B) */
}

void har_block2_forward(const float *x64, float *y64,
                  float *s64_0, float *s64_1) {
   har_block2_branch_main(x64, s64_0, s64_1);     /* A -> s64_1 */
   har_residual_merge_relu(s64_1, x64, 64 * HAR_FILTERS, y64); /* add(A,B) */
}

void har_head_forward(const float *x64, float *logits) {
   float pooled[HAR_FILTERS];

   har_global_max_pool(x64, 64, HAR_FILTERS, pooled);
   har_dense(pooled, HAR_DENSE_IN, har_activity_logits_w, har_activity_logits_b,
           HAR_DENSE_OUT, logits);
}

void har_resnet_predict(const float *x_norm, float *logits) {
   har_block1_forward(x_norm, buf_c, buf_a, buf_b, buf_d);
   har_block2_forward(buf_c, buf_c, buf_e, buf_d);
   /* Head: (64,32) -> logits(6). */
   har_head_forward(buf_c, logits);
}
```

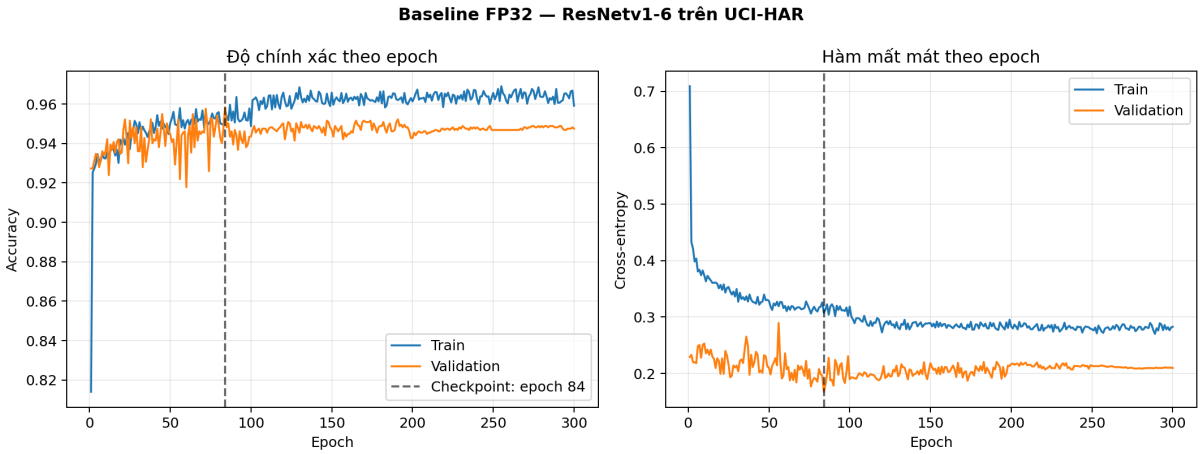
6 Kết quả và thảo luận

6.1 Baseline FP32

Checkpoint được chọn ở epoch $84/300$: train accuracy 95,64%, validation accuracy 95,82%. Trên test chính thức baseline đạt accuracy 92,6026% và macro-F1 0,9271 ($2.729/2.947$ mẫu đúng). Validation accuracy ở epoch 300 là 94,75%, thấp hơn mức 95,82% của checkpoint được chọn, nên chọn theo validation là hợp lý; đồng thời vì checkpoint được chọn chính bằng validation, con số validation là ước lượng lạc quan và test mới là con số báo cáo (Hình 4). Dung lượng `baseline.keras` là 148.495 byte.

6.2 Ảnh hưởng của kích thước calibration (validation)

Bảng 5 và Hình 5 cho thấy accuracy trung bình tăng nhẹ theo n (từ $95{,}6678\%$ lên $95{,}7576\%$) và $n = 500$ được chọn. Tuy nhiên chênh lệch giữa ba cấu hình nhỏ hơn độ lệch chuẩn giữa các seed ($0{,}10$–$0{,}18$ điểm %), nên kết luận phù hợp là $n = 500$ tốt nhất *trong phạm vi ba seed đã chạy*, không khẳng định luôn tốt nhất. Đáng chú ý, ở $n = 50$ seed 33 cho validation accuracy $95{,}8249\%$, trùng đúng FP32; biến thiên giữa các seed (Bảng 7) vì vậy lớn hơn tác động trung bình của n.



Hình 4. Accuracy và loss theo epoch của baseline; đường nét đứt đánh dấu checkpoint ở epoch 84. Learning rate giảm tại epoch 100, 200, 250.

Mô hình	Số lần chạy	Accuracy (%)	Macro-F1	Dung lượng (byte)
FP32 (TFLite)	1	95,8249	0,9621	51.728
INT8, $n = 50$	3	95,6678 $\pm$ 0,1695	0,9607 $\pm$ 0,0015	25.104
INT8, $n = 200$	3	95,7127 $\pm$ 0,1029	0,9611 $\pm$ 0,0009	25.104
INT8, $n = 500$	3	**95,7576 $\pm$ 0,1782**	**0,9614 $\pm$ 0,0016**	25.104

Bảng 5. Kết quả validation, trung bình $\pm$ độ lệch chuẩn mẫu qua ba seed (in đậm: cấu hình được chọn).

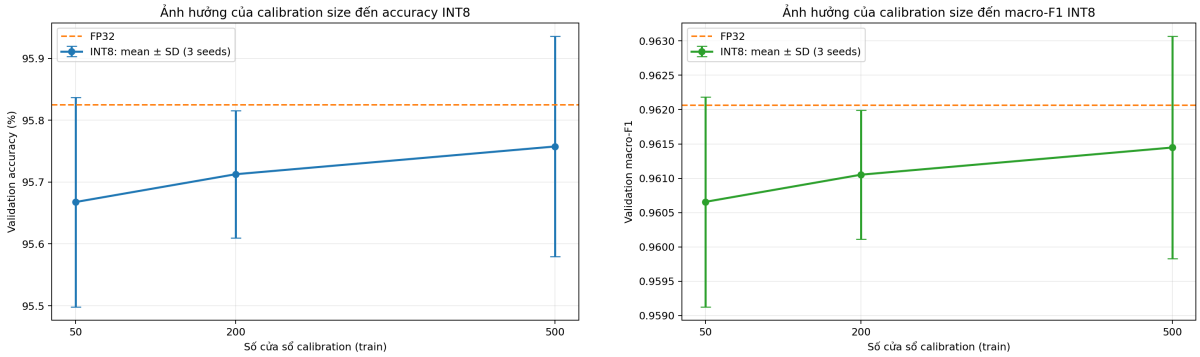
6.3 So sánh FP32 và INT8 trên test

Bảng 6 là kết quả cuối cùng, đo sau khi chốt $n = 500$. INT8 đạt 92,1841 $\pm$ 0,2722% và macro-F1 0,9232, thấp hơn FP32 0,4185 điểm % accuracy; số mẫu sai tăng từ 218 (FP32) lên 222–238 (INT8), tức thêm 4 đến 20 mẫu trên 2.947 (Bảng 7). FP32 và INT8 được đánh giá trên cùng tập test, nên kiểm định ghép cặp trên dự đoán từng mẫu (ví dụ McNemar) mới cho kết luận chặt chẽ về ý nghĩa thống kê; kho kết quả hiện chỉ lưu ma trận nhầm lẫn nên nhóm chưa thực hiện. Để lấy thang so sánh, độ bất định lấy mẫu nhị thức của chính con số accuracy FP32 trên 2.947 mẫu cỡ $\pm$0,48 điểm %, lớn hơn mức giảm trung bình 0,42 điểm %.

Theo từng lớp. Bảng 8 và Hình 6 cho thấy lỗi tập trung ở cặp hoạt động tĩnh `SITTING`/`STANDING`: ở FP32, hai chiều nhầm lẫn này chiếm 131 trên 218 mẫu sai (60,1%). Lượng tử hóa làm giảm recall của hai lớp này nhiều nhất (`STANDING` $-1{,}50$ và `SITTING` $-1{,}09$ điểm %), trong khi các lớp chuyển động và `LAYING` gần như không đổi. Điều này phù hợp với hai hoạt động tĩnh có tín hiệu quán tính rất giống nhau, vốn đã là điểm khó của FP32; lượng tử hóa không đổi bản chất điểm khó mà làm nó nặng thêm một chút (Bảng 9).

6.4 Dung lượng mô hình

File FP32 TFLite là 51.728 byte, INT8 là 25.104 byte (cả chín mô hình INT8 bằng nhau), giảm 51,47%, hệ số nén 2,06 lần (Hình 7). Con số này *không* đạt bốn lần dù trọng số xuống 8 bit, vì mạng nhỏ và file còn chứa đồ thị, shape, tham số lượng tử và metadata cố định. Cụ thể, riêng trọng số FP32 là $10.726 \times 4 = 42.904$ byte, còn INT8 là 10.726 byte (đúng bốn lần); phần còn lại trong file là $51.728 - 42.904 = 8.824$ byte (FP32) và $25.104 - 10.726 = 14.378$ byte (INT8). Phần dư



Hình 5. Validation accuracy (trái) và macro-F1 (phải) của INT8 theo số cửa sổ calibration; đường nét đứt là FP32, thanh sai số là độ lệch chuẩn qua ba seed.

Mô hình	Accuracy (%)	Δ so với FP32 (điểm %)	Macro-F1	Dung lượng (byte)
FP32 (TFLite)	92,6026	0,0000	0,9271	51.728
INT8, $n = 500$ (3 seed)	92,1841 ± 0,2722	−0,4185	0,9232 ± 0,0026	25.104

Bảng 6. Kết quả test: FP32 TFLite và INT8 $n = 500$ (trung bình ± độ lệch chuẩn qua 3 seed).

lớn hơn ở INT8 phù hợp với việc mỗi tensor lượng tử hóa mang thêm scale/zero-point (theo kênh với trọng số) và bias lưu 32 bit; nhóm không phân rã chính xác từng byte của file. Mô hình INT8 gồm 23 toán tử (5 `CONV_2D`, 2 `MAX_POOL_2D`, 2 `ADD`, 1 `REDUCE_MAX`, 1 `FULLY_CONNECTED`, cùng 6 `EXPAND_DIMS` và 6 `RESHAPE`, nghĩa là Conv1D được hiện thực bằng Conv2D); ReLU không xuất hiện thành toán tử riêng, thường do TFLite gộp vào toán tử đứng trước. Đây là dung lượng file, không phải Flash/RAM thực đo trên vi điều khiển.

6.5 Kết quả trích xuất tham số

Từ checkpoint FP32, `export-c` trích đủ 10.726 tham số; Bảng 10 cho cấu trúc, Bảng 11 cho thống kê, và các bảng sau đưa ra giá trị cụ thể của một số tensor. Toàn bộ thống kê được tính lại từ `params_fp32.npz` và khớp `results/parameters_reference.md`.

Tham số chuẩn hóa đầu vào. Bảng 12 liệt kê μ_c, σ_c của chín kênh, được nhúng vào mã C (`har_input_mean`, `har_input_std`). Kênh gia tốc tổng theo trục x có trung bình $\approx 0{,}81$ vì chứa thành phần trọng lực, còn các kênh `body_*` có trung bình gần 0.

Trọng số lớp phân loại. Bảng 13 là toàn bộ $32 \times 6 + 6 = 198$ tham số của `activity_logits`: cột o là trọng số nối 32 đặc trưng sau GlobalMaxPool tới logit của lớp thứ o; hàng cuối là bias.

Ví dụ trích từ lớp Conv đầu. Bảng 14 là kernel của filter đầu tiên (đầu ra $c_o = 0$) của `block1_conv1`: ba vị trí thời gian k nhân với chín kênh vào. Kernel đầy đủ của lớp chiếu nhánh B ($1 \times 9 \times 32$) nằm ở Phụ lục A.

Tham số lượng tử hóa INT8. Ngoài trọng số FP32, mỗi mô hình INT8 mang tham số affine (8) cho đầu vào và đầu ra (Bảng 15). Scale và zero-point thay đổi theo tập calibration, nên ba seed cho ba bộ giá trị khác nhau với cùng một checkpoint; đây là nguồn biến thiên của kết quả INT8. Cả chín mô hình đều full-integer (không còn tensor float).

Tệp C sinh tự động và bộ nhớ tham số. Bảng 16 cho kích thước và SHA-256 của các tệp

Tập	Mô hình	Seed	Accuracy (%)	Macro-F1	Số mẫu đúng	Số mẫu sai
Validation	FP32	–	95,8249	0,9621	1.423	62
Validation	INT8 $n = 50$	11	95,6902	0,9609	1.421	64
Validation	INT8 $n = 50$	22	95,4882	0,9590	1.418	67
Validation	INT8 $n = 50$	33	95,8249	0,9621	1.423	62
Validation	INT8 $n = 200$	11	95,8249	0,9621	1.423	62
Validation	INT8 $n = 200$	22	95,6902	0,9608	1.421	64
Validation	INT8 $n = 200$	33	95,6229	0,9602	1.420	65
Validation	INT8 $n = 500$	11	95,6229	0,9602	1.420	65
Validation	INT8 $n = 500$	22	95,9596	0,9633	1.425	60
Validation	INT8 $n = 500$	33	95,6902	0,9608	1.421	64
Test	FP32	–	92,6026	0,9271	2.729	218
Test	INT8 $n = 500$	11	91,9240	0,9208	2.709	238
Test	INT8 $n = 500$	22	92,4669	0,9259	2.725	222
Test	INT8 $n = 500$	33	92,1615	0,9228	2.716	231

Bảng 7. Kết quả từng mô hình và từng seed (nguồn: `results/metrics.csv`); số mẫu đúng suy ra từ accuracy.

Lớp	Số mẫu	Recall FP32	Recall INT8	Precision FP32	Precision INT8	Δ recall (điểm %)
WALKING	496	95,56	95,50	95,95	95,95	−0,07
WALKING_UPSTAIRS	471	92,57	92,92	98,42	98,43	+0,35
WALKING_DOWNSTAIRS	420	95,95	96,03	95,27	95,13	+0,08
SITTING	491	82,89	81,81	83,06	81,92	−1,09
STANDING	532	90,23	88,72	84,66	83,55	−1,50
LAYING	537	98,51	98,39	99,81	99,81	−0,12

Bảng 8. Recall và precision theo lớp trên test; INT8 là trung bình ba seed ($n = 500$).

C được sinh; đoạn dưới đây là phần đầu của mảng trọng số `block1_conv1` đúng như trong mã (hai dòng giá trị đầu, đối chiếu khớp `params_fp32.npz`). Bộ nhớ tham số lý thuyết là 42.904 byte ở FP32 và 10.726 byte nếu mọi tham số lưu 8 bit; con số này khác dung lượng file TFLite đo ở trên.

Mã 3. Đầu mảng `har_block1_conv1_w` trong `har_resnet_params.c`.

```
/* shape (3, 9, 32) -> flatten [k][cin][cout] */
const float har_block1_conv1_w[HAR_BLOCK1_CONV1_K * HAR_BLOCK1_CONV1_CIN * HAR_BLOCK1_CONV1_COUT] = {
    0.621398985f, 0.531395614f, -0.135080755f, 0.0896557122f, -0.165028855f, -0.276271641f, 0.674555659f, -0.368312627f,
    0.00433768006f, -0.126328662f, -0.247460112f, -0.0871876031f, 0.0896287262f, 0.0443048738f, -0.151708812f, 0.12130516f,
```

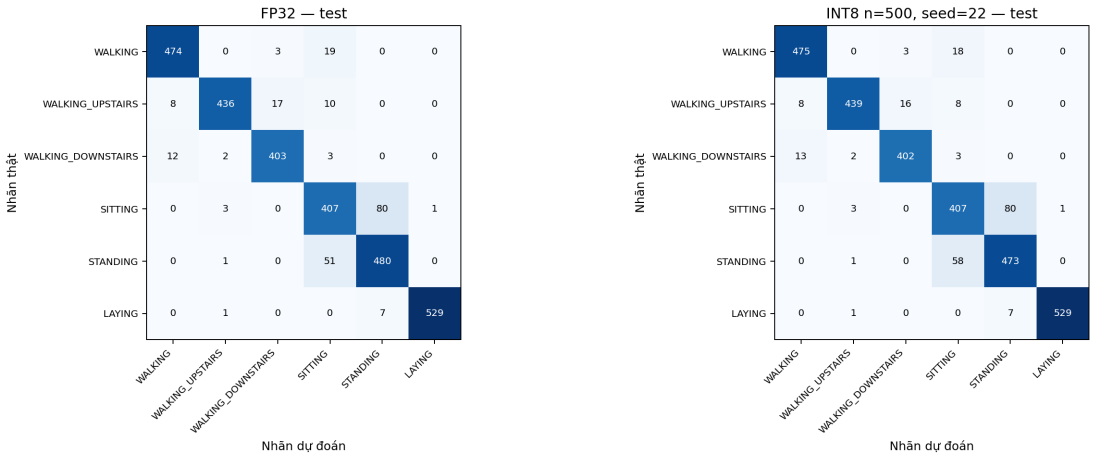
6.6 Kết quả suy luận bằng C

Mã C được đối chiếu với Keras trên 12 cửa sổ validation (hai cửa sổ mỗi lớp) đã lưu cùng logits của Keras:

- Tham chiếu NumPy (`reference_forward.py`, cùng thuật toán với C) so với Keras: sai số logits lớn nhất $1{,}9 \times 10^{-6}$, argmax khớp 12/12.
- Chương trình C: argmax khớp 12/12, sai số logits lớn nhất cỡ 10^{-6} (ngưỡng chấp nhận 10^{-3}); hàm `har_normalize` sai số $\leq 6 \times 10^{-8}$; kiểm tra ba hàm khối chạy tuần tự so với `har_resnet_predict` nguyên khối cho kết quả *bằng nhau tuyệt đối*.
- Biên dịch C99 `-Wall -Wextra -Wpedantic`: 0 cảnh báo. Bộ kiểm thử tự động của repository (`python -m pytest -q`) hiện đạt 37/37 test.
- Bộ nhớ: buffer trung gian tĩnh 57.344 byte; cộng đầu vào 4.608 byte, mảng cục bộ 128 byte

Mô hình	SITTING→STANDING	STANDING→SITTING	Tổng số mẫu sai
FP32	80	51	218
INT8 $n = 500$, seed 11	89	59	238
INT8 $n = 500$, seed 22	80	58	222
INT8 $n = 500$, seed 33	87	60	231

Bảng 9. Số mẫu bị nhầm giữa SITTING và STANDING trên test (hàng: nhãn thật).



Hình 6. Ma trận nhầm lẫn trên test: FP32 (trái) và INT8 $n = 500$, seed 22 (phải, mô hình đại diện được chọn trong results/selection.json).

và logits 24 byte, RAM hoạt động ước lượng 62.104 byte; bảng trọng số (42.904 byte) là const nên nằm ở vùng chỉ đọc. Mỗi cửa sổ cần 927.360 phép nhân–cộng.

Các con số bộ nhớ và MAC là *ước lượng/đếm theo thiết kế*; chưa đo trên vi điều khiển, và chưa đo accuracy của bản C trên toàn tập test (2.947 cửa sổ).

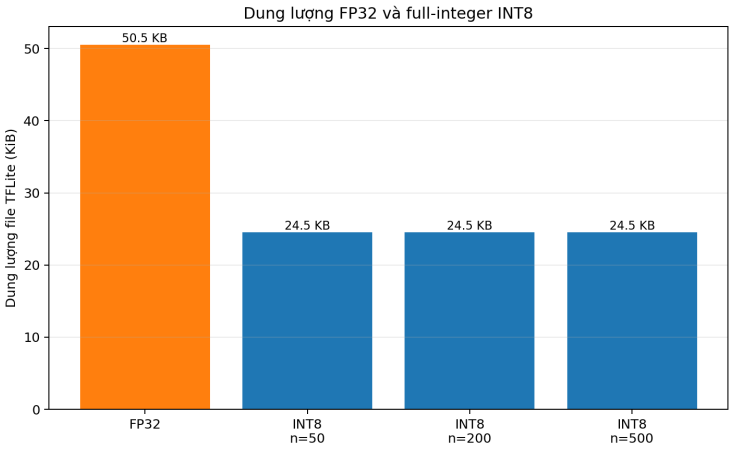
6.7 Thảo luận

Đánh đổi. Với cấu hình khảo sát, PTQ INT8 cho file nhỏ đi hơn một nửa với mức giảm accuracy dưới 0,5 điểm %, và mức giảm này tập trung vào cặp SITTING/STANDING. **Hệ số nén.** Hệ số 2,06 lần thấp hơn mức lý thuyết bốn lần vì phần dư cố định của định dạng file lớn so với mạng chỉ có 10.726 tham số. **Độ tin cậy.** Chỉ có một baseline (seed 42), ba seed calibration và một tập test; khác biệt giữa các n nằm trong nhiễu giữa các seed.

Giới hạn. (i) Một kiến trúc, một bộ dữ liệu, ba giá trị n. (ii) Không đo độ trễ, RAM, Flash, năng lượng; không suy từ laptop. (iii) Kết quả là TFLite PTQ INT8, không so trực tiếp với MicroAI/QAT của bài báo. (iv) Mã C mới là FP32 tham chiếu; INT8 trong C chưa làm. (v) Chưa có kiểm định thống kê ghép cặp giữa FP32 và INT8.

7 Kết luận và hướng phát triển

Báo cáo đã (1) xây dựng baseline FP32 ResNetv1-6 một chiều trên UCI-HAR đạt 92,60% accuracy và macro-F1 0,9271 trên test; (2) lượng tử hóa PTQ full-integer INT8, chọn $n = 500$ bằng validation, đạt 92,1841 $\pm$ 0,2722% trên test với file nhỏ hơn 51,47%; (3) mô tả luồng và



Hình 7. Dung lượng file TFLite: FP32 và INT8 với ba kích thước calibration.

#	Layer	Khối	Nhánh	Phép toán	Shape kernel	Số tham số	% tổng
1	`block1_conv1`	Khối 1	A (chính)	Conv1D, K=3, same	(3, 9, 32)	896	8,35
2	`block1_conv2`	Khối 1	A (chính)	Conv1D, K=3, same	(3, 32, 32)	3.104	28,94
3	`block1_projection`	Khối 1	B (shortcut)	Conv1D, K=1, valid	(1, 9, 32)	320	2,98
4	`block2_conv1`	Khối 2	A (chính)	Conv1D, K=3, same	(3, 32, 32)	3.104	28,94
5	`block2_conv2`	Khối 2	A (chính)	Conv1D, K=3, same	(3, 32, 32)	3.104	28,94
6	`activity_logits`	Đầu ra	–	Dense $32 \to 6$	(32, 6)	198	1,85
					Tổng	**10.726**	**100,00**

Bảng 10. Tham số đã trích xuất theo layer. Bốn lớp Conv K=3 ở nhánh chính chiếm 95,17% tổng tham số; lớp chiếu nhánh B chỉ 2,98% và lớp Dense 1,85%.

thuật toán của mô hình theo từng khối và từng nhánh bằng công thức; (4) trích xuất toàn bộ 10.726 tham số kèm thống kê và giá trị cụ thể; (5) cài đặt suy luận C99 thuần khớp với Keras trên 12 cửa sổ đối chiếu.

Hướng phát triển: cài INT8 trong C (tích lũy `int32`, requantize bằng multiplier và shift) và xuất trọng số INT8; port sang vi điều khiển rồi đo RAM, Flash, độ trễ và năng lượng thật; đo accuracy của bản C trên toàn tập test; thử QAT hoặc nhiều seed huấn luyện hơn để so sánh công bằng với bài báo; kiểm định ghép cặp FP32 và INT8.

Tài liệu tham khảo

[1] P.-E. Novac, G. Boukli Hacene, A. Pegatoquet, B. Miramond, V. Gripon. Quantization and Deployment of Deep Neural Networks on Microcontrollers. *Sensors*, 21(9):2984, 2021. `https://doi.org/10.3390/s21092984`

[2] D. Anguita, A. Ghio, L. Oneto, X. Parra, J. L. Reyes-Ortiz. A Public Domain Dataset for Human Activity Recognition Using Smartphones. *ESANN*, 2013. Dữ liệu: `https://doi.org/10.24432/C54S4K`

[3] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 770–778, 2016.

Layer	Tensor	Số phần tử	min	max	trung bình	độ lệch chuẩn
`block1_conv1`	kernel	864	−0,7666	+0,7822	+0,0014	0,2509
	bias	32	−0,8689	+0,5869	−0,1822	0,3396
`block1_conv2`	kernel	3.072	−0,6782	+0,4601	−0,0376	0,1425
	bias	32	−0,8619	+0,6701	+0,0708	0,2806
`block1_projection`	kernel	288	−1,0266	+1,2403	+0,0169	0,3021
	bias	32	−0,8619	+0,6701	+0,0708	0,2806
`block2_conv1`	kernel	3.072	−0,5980	+0,5741	−0,0244	0,1439
	bias	32	−0,5400	+0,5516	−0,0289	0,2893
`block2_conv2`	kernel	3.072	−0,5754	+0,5147	−0,0257	0,1335
	bias	32	−0,5410	+1,2143	−0,0412	0,3124
`activity_logits`	kernel	192	−1,3688	+0,9113	+0,0186	0,3290
	bias	6	−1,0907	+1,3195	0,0000	1,0229

Bảng 11. Thống kê số học của từng tensor (kernel, bias). Không có trọng số bằng đúng 0 ở layer nào.

#	Kênh	μ_c	σ_c
0	`total_acc_x`	+0,805391	0,410322
1	`total_acc_y`	+0,029006	0,399567
2	`total_acc_z`	+0,080120	0,354005
3	`body_gyro_x`	+0,004766	0,408725
4	`body_gyro_y`	−0,002092	0,382862
5	`body_gyro_z`	−0,000882	0,258296
6	`body_acc_x`	−0,000794	0,197001
7	`body_acc_y`	−0,000252	0,126466
8	`body_acc_z`	−0,000306	0,108309

Bảng 12. Tham số Z-score theo kênh (fit trên tập train mô hình).

[4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. *CVPR*, 2704–2713, 2018.

[5] R. Krishnamoorthi. Quantizing deep convolutional networks for efficient inference: A whitepaper. *arXiv:1806.08342*, 2018.

[6] H. Zhang, M. Cisse, Y. N. Dauphin, D. Lopez-Paz. mixup: Beyond Empirical Risk Minimization. *ICLR*, 2018.

A Kernel đầy đủ của lớp chiếu nhánh B

Bảng 17 là toàn bộ $1 \times 9 \times 32 + 32 = 320$ tham số của `block1_projection` (Conv1D $K = 1$, đổi 9 kênh cảm biến thành 32 kênh): hàng c_o là một filter, các cột c_i là trọng số tương ứng chín kênh vào theo thứ tự `total_acc_x`, …, `body_acc_z`; cột cuối là bias.

B Tái lập kết quả

Mã nguồn và kết quả nằm tại `https://github.com/Royce-23/quantization`. Thứ tự lệnh (môi trường Python 3.10.12, TensorFlow CPU 2.16.1, NumPy 1.26.4):

i	$W_{i,0}$ (WALK)	$W_{i,1}$ (UP)	$W_{i,2}$ (DOWN)	$W_{i,3}$ (SIT)	$W_{i,4}$ (STAND)	$W_{i,5}$ (LAY)
0	+0,1220	−0,1532	+0,1674	−0,1572	+0,5603	−0,2510
1	+0,2476	+0,5378	+0,0287	−0,1578	+0,2236	+0,1310
2	−0,1774	−0,3461	+0,5742	−0,2613	−0,1697	−0,0609
3	+0,6200	−0,1224	−0,0557	−0,1071	−0,1122	+0,1811
4	+0,0218	+0,5228	+0,1563	+0,0911	+0,0705	+0,1851
5	+0,0747	−0,0015	+0,0163	+0,6488	−0,2407	−0,1888
6	+0,2125	−0,2097	−0,1106	−0,1618	−0,3441	−0,1183
7	+0,3183	+0,1543	+0,2512	−0,0529	+0,3270	+0,0025
8	+0,1319	−0,1722	+0,1796	+0,0068	−0,2766	+0,0082
9	+0,1553	−0,2212	+0,1969	+0,2753	+0,1306	+0,1299
10	+0,5160	+0,3572	+0,1914	+0,2755	+0,6152	−1,3688
11	+0,0489	−0,4124	−0,2870	+0,4899	+0,1995	−0,0337
12	−0,0392	+0,2016	−0,0930	−0,4065	+0,4382	−0,0663
13	+0,2244	−0,0893	+0,1855	−0,2424	+0,5357	−0,4969
14	+0,2307	+0,1067	+0,0512	+0,3517	−0,4180	+0,1631
15	−0,2092	−0,1823	+0,0260	−0,0572	−0,3345	+0,9113
16	−0,2471	−0,1259	−0,1040	+0,6953	+0,0535	+0,0012
17	−0,1196	+0,0813	+0,0173	+0,2682	−0,6713	−0,2553
18	−0,5144	−0,4805	−0,6201	+0,7217	+0,6369	+0,2566
19	−0,1813	−0,0103	−0,0919	−0,4555	+0,4471	+0,0214
20	−0,2525	−0,3858	−0,0506	+0,5644	+0,1174	−0,2859
21	−0,2676	−0,2568	+0,7880	−0,5745	−0,4911	−0,2306
22	+0,1406	+0,2201	+0,1340	−0,2082	−0,5124	+0,2720
23	−0,2808	+0,5147	−0,1133	+0,0725	−0,3113	−0,0571
24	+0,2401	+0,3392	+0,1145	−0,4053	−0,0572	+0,2075
25	+0,1233	+0,0673	+0,2419	−0,3966	−0,0835	−0,5704
26	+0,1420	+0,2557	−0,0044	−0,3969	−0,2923	+0,1722
27	+0,2526	−0,2549	−0,0944	+0,1491	+0,4079	+0,1172
28	+0,8616	+0,1610	−0,0753	−0,0982	−0,4140	−0,0064
29	+0,0534	+0,4234	+0,4287	+0,0288	−0,2182	−0,8381
30	−0,2320	+0,7949	−0,0168	−0,0004	−0,2621	−0,0957
31	+0,0224	+0,3162	+0,1494	−0,2963	+0,2266	−0,2956
b_o	−0,9656	−0,8051	−1,0907	+1,3195	+1,3083	+0,2336

Bảng 13. Toàn bộ trọng số và bias của lớp `activity_logits` (các cột theo thứ tự nhãn `WALKING`, …, `LAYING`).

k	c_i=0	c_i=1	c_i=2	c_i=3	c_i=4	c_i=5	c_i=6	c_i=7	c_i=8
0	+0,621	−0,767	−0,058	+0,182	+0,071	−0,111	−0,328	−0,068	+0,165
1	+0,609	−0,649	+0,177	−0,111	−0,024	+0,097	−0,358	+0,234	+0,028
2	+0,443	−0,649	+0,121	−0,302	−0,056	+0,052	−0,098	+0,343	−0,037

Bảng 14. Kernel $W_{k,c_i,0}$ của filter 0 trong `block1_conv1` (hình dạng 3×9).

```
python main.py prepare  # tai va chuan bi UCI-HAR
python main.py train    # huan luyen baseline FP32 (300 epoch)
python main.py quantize # 1 FP32 TFLite + 9 INT8 TFLite
python main.py evaluate # validation -> chon n -> test
python main.py summarize ; python main.py plot
python main.py export-c # tham so + ma C + golden vector
python main.py describe-params # results/parameters_reference.md
cd firmware && make test # build C99 va doi chieu 12 golden vector
python report/scripts/gen_tables.py # sinh bang/so lieu cua bao cao
```

Các bảng và số liệu trong báo cáo được sinh từ `results/*.csv`, `models/*.json` và `params_fp32.npz` bởi `report/scripts/gen_tables.py`, nên không gõ tay.

Mô hình	Scale vào	Zero-point vào	Scale ra	Zero-point ra	Số tensor float
INT8 $n = 50$, seed 11	0.069558	-6	0.048779	-33	0
INT8 $n = 50$, seed 22	0.065165	9	0.048235	-35	0
INT8 $n = 50$, seed 33	0.077086	1	0.052187	-28	0
INT8 $n = 200$, seed 11	0.086882	-26	0.052371	-32	0
INT8 $n = 200$, seed 22	0.088959	14	0.053451	-38	0
INT8 $n = 200$, seed 33	0.077859	21	0.054190	-46	0
INT8 $n = 500$, seed 11	0.091455	-2	0.054448	-30	0
INT8 $n = 500$, seed 22	0.087212	-4	0.059316	-28	0
INT8 $n = 500$, seed 33	0.088829	4	0.055678	-27	0

Bảng 15. Scale và zero-point của đầu vào/đầu ra các mô hình INT8 (nguồn: `models/model_metadata.json`).

Tệp sinh tự động	Byte	SHA-256
`include/har_resnet_params.h`	2.113	`a24805c6ec09515b144d425f1e4fa5d761548f1cf8c499003e4b20506e0dada0`
`src/har_resnet_params.c`	168.839	`75aa6d044e1c3efe9551ceadfe12dadcaccadcd997dc9a7f9769b65596402585`
`include/har_golden.h`	537	`871b27be97d60ed8d9dd7e667a22942bcd4b3a574278389e30ff4a65c850e572`
`src/har_golden.c`	209.743	`81904accaae525a4290643433c0649d9bec0205e902d538cf821f0102af4345e`

Bảng 16. Tệp C sinh tự động bởi `export-c` (nguồn: `models/params_manifest.json`).

c_o	c_i=0	c_i=1	c_i=2	c_i=3	c_i=4	c_i=5	c_i=6	c_i=7	c_i=8	b
0	+0,879	−0,538	−0,510	−0,725	−0,141	−0,242	−0,416	+0,111	+0,198	−0,143
1	−0,294	+0,340	−0,121	+0,409	−0,040	−0,033	−0,117	+0,528	−0,423	+0,061
2	−0,164	+0,326	+0,036	+0,317	+0,177	+0,015	+0,231	−0,249	−0,119	+0,103
3	−0,398	+0,184	−0,033	−0,266	+0,012	−0,338	−0,476	−0,111	−0,612	+0,141
4	−0,064	+0,172	+0,536	+0,577	+0,073	−0,040	−0,379	+0,066	+0,110	+0,025
5	+1,240	+0,627	+0,376	−0,076	−0,186	+0,285	−0,024	+0,146	+0,114	+0,201
6	−0,408	−0,213	+0,201	−0,038	+0,749	−0,211	−0,359	−0,307	+0,058	+0,282
7	−0,125	−0,163	−0,048	+0,084	+0,402	+0,172	−0,016	+0,220	−0,095	−0,024
8	−0,173	+0,297	−0,248	+0,345	+0,101	+0,711	+0,050	−0,122	−0,356	−0,032
9	+0,164	+0,418	−0,701	−0,291	−0,104	−0,349	−0,082	−0,018	+0,139	+0,153
10	+0,801	−0,667	−0,464	−0,061	+0,143	+0,278	−0,557	−0,085	−0,057	+0,061
11	−0,082	−0,141	+0,277	−0,023	+0,034	+0,089	+0,008	+0,201	+0,028	−0,106
12	−0,038	−1,027	−0,176	−0,049	+0,140	−0,236	−0,005	+0,383	−0,016	−0,017
13	+0,565	−0,017	−0,043	−0,018	+0,238	−0,119	−0,164	+0,021	−0,149	+0,077
14	+0,252	+0,390	+0,661	−0,051	+0,103	−0,281	+0,199	+0,194	+0,092	−0,124
15	−0,815	+0,784	+0,481	+0,248	+0,044	+0,131	+0,309	−0,095	−0,078	+0,553
16	+0,546	+0,599	+0,385	−0,229	−0,264	+0,061	−0,273	−0,065	−0,069	+0,126
17	+0,575	+0,310	+0,123	−0,155	+0,314	−0,716	−0,365	+0,170	+0,155	+0,206
18	−0,244	−0,189	+0,070	0,000	+0,087	+0,012	−0,229	+0,122	+0,106	−0,144
19	−0,479	+0,073	+0,121	+0,566	+0,340	+0,095	−0,248	−0,042	+0,031	+0,670
20	+0,101	−0,262	+0,130	+0,031	+0,050	+0,240	−0,059	+0,174	+0,093	+0,265
21	+0,383	+0,278	+0,143	−0,047	+0,218	−0,412	+0,171	−0,121	−0,018	+0,087
22	−0,741	+0,080	+0,232	+0,086	+0,048	−0,401	+0,186	+0,323	+0,333	−0,862
23	−0,238	−0,115	+0,093	−0,044	−0,169	−0,206	−0,222	+0,500	−0,280	+0,148
24	−0,876	+0,052	+0,068	−0,124	−0,038	+0,376	−0,335	−0,195	+0,123	+0,650
25	+0,197	−0,300	−0,044	−0,219	+0,042	+0,099	−0,185	+0,067	+0,060	−0,272
26	−0,376	+0,199	−0,111	+0,146	−0,097	−0,238	−0,329	+0,014	+0,098	−0,378
27	+0,049	+0,032	+0,120	+0,181	+0,027	+0,087	−0,039	+0,142	+0,223	+0,152
28	+0,153	+0,371	+0,012	+0,276	+0,178	+0,242	+0,176	−0,480	+0,167	+0,293
29	+0,322	−0,086	−0,333	+0,074	−0,011	+0,203	+0,295	−0,328	−0,128	+0,005
30	−0,004	+0,142	+0,079	+0,057	−0,140	−0,052	+0,378	−0,260	+0,271	+0,126
31	+0,205	−0,173	−0,559	+0,465	+0,193	−0,028	+0,073	−0,071	+0,214	−0,019

Bảng 17. Kernel W_{0,c_i,c_o} và bias của `block1_projection`.